

Pytest Concepts

with Detailed Examples
Testing | Fixtures | Mocking | Plugins

Table of Contents

1. Installation & Basic Test Structure
2. Running Tests — CLI Options
3. Assertions
4. Fixtures
5. Fixture Scope & Teardown
6. `conftest.py` & Fixture Sharing
7. Parametrization
8. Markers
9. Testing Exceptions
10. Mocking & Patching
11. Testing Class-Based Code
12. Temporary Files & Directories
13. Capturing Output & Logs
14. Plugins
15. Test Organization
16. Coverage
17. Async Testing
18. Configuration Files

19. Hooks

20. Best Practices

1. Installation & Basic Test Structure

Pytest is the most popular Python testing framework — simple syntax, powerful fixtures, and a huge plugin ecosystem.

Installation

```
pip install pytest
```

```
# Check version
pytest --version
```

Your First Test

```
# test_math.py
def add(a, b):
    return a + b

def test_add():
    assert add(2, 3) == 5

def test_add_negative():
    assert add(-1, -1) == -2

# Run with: pytest test_math.py
```

Naming Conventions

Item	Convention
Test files	test_*.py or *_test.py
Test functions	def test_*(): ...
Test classes	class Test* (no __init__)
Test methods	def test_* inside Test class

```
# Class-based tests (grouping related tests)
class TestCalculator:
    def test_add(self):
        assert add(2, 3) == 5

    def test_subtract(self):
        assert add(5, -2) == 3
```

■ **Note:** Pytest auto-discovers files matching test_*.py / *_test.py and functions/methods starting with test_.

2 Running Tests - CLI Options

```
pytest                                # run all tests in current dir
pytest test_math.py                  # run a specific file
pytest test_math.py::test_add        # run a specific test function
pytest test_math.py::TestCalculator::test_add # specific method

pytest -v                            # verbose output
pytest -q                            # quiet output
pytest -x                            # stop after first failure
pytest --maxfail=3                   # stop after 3 failures
pytest -k 'add'                      # run tests matching name 'add'
pytest -k 'add and not negative'     # expression-based filtering
pytest -m slow                       # run tests marked @pytest.mark.slow

pytest --lf                          # run last failed tests
pytest --ff                          # run failed first, then rest
pytest -s                            # show print() output (disable capture)
pytest --tb=short                    # shorter traceback format
pytest --tb=line                     # one-line traceback
pytest -n auto                       # parallel run (pytest-xdist)
pytest --durations=10                # show 10 slowest tests
pytest --collect-only                # list tests without running
pytest -rA                           # show summary of all results
```

✓ **Tip:** Use 'pytest -k' for quick filtering during development instead of remembering exact paths.

3. Assertions

Pytest uses Python's plain `assert` statement and rewrites it to give rich failure diagnostics — no special assert methods needed.

```
def test_examples():
    assert 1 + 1 == 2
    assert 'py' in 'pytest'
    assert [1, 2, 3] == [1, 2, 3]
    assert {'a': 1} == {'a': 1}
    assert isinstance(5, int)
    assert not False

    # Custom failure message
    x = 5
    assert x > 0, f'Expected positive, got {x}'

    # Float comparison (avoid == with floats)
    from pytest import approx
    assert 0.1 + 0.2 == approx(0.3)
    assert 100.0 == approx(99.999, rel=1e-2)
```

Failure Output Example

```
def test_fails():
    assert [1, 2, 3] == [1, 2, 4]

# pytest auto-generates a detailed diff:
# E      assert [1, 2, 3] == [1, 2, 4]
# E          At index 2 diff: 3 != 4
# E      Full diff:
# E      - [1, 2, 4]
# E      + [1, 2, 3]
```

✓ **Tip:** Never write custom assert helper classes like unittest's `assertEqual` — plain `assert` + pytest's introspection is enough.

4. Fixtures

Fixtures provide reusable setup/teardown code. Test functions simply declare a fixture as a parameter and pytest injects it.

Basic Fixture

```
import pytest

@pytest.fixture
def sample_list():
    return [1, 2, 3, 4, 5]

def test_length(sample_list):
    assert len(sample_list) == 5

def test_sum(sample_list):
    assert sum(sample_list) == 15
```

Fixture with Setup & Teardown (yield)

```
@pytest.fixture
def db_connection():
    conn = create_connection() # setup
    yield conn                 # provided to the test
    conn.close()               # teardown – always runs

def test_query(db_connection):
    result = db_connection.execute('SELECT 1')
    assert result is not None
```

Fixtures Using Other Fixtures

```
@pytest.fixture
def user():
    return {'name': 'Alice', 'role': 'admin'}

@pytest.fixture
def auth_token(user): # depends on 'user' fixture
    return generate_token(user['name'])

def test_token_valid(auth_token):
    assert verify_token(auth_token)
```

Built-in Fixtures

Fixture	Purpose
tmp_path	Unique temporary directory (pathlib.Path)
tmp_path_factory	Create multiple temp directories
capsys	Capture stdout/stderr
caplog	Capture logging output
monkeypatch	Modify objects/env vars safely (auto-undo)

request	Access test context/metadata
pytestconfig	Access pytest configuration

5. Fixture Scope & Teardown

Scope Levels

Scope	Created Once Per	Use Case
function (default)	Each test function	Isolated state per test
class	Each test class	Shared setup within a class
module	Each .py test file	Expensive setup per file
package	Each package	Setup shared by a package
session	Entire test run	DB connections, servers

```
@pytest.fixture(scope='session')
def api_server():
    server = start_test_server()    # expensive – run once
    yield server
    server.stop()

@pytest.fixture(scope='module')
def shared_data():
    return load_large_dataset()

@pytest.fixture(scope='function')    # default – fresh per test
def fresh_list():
    return []
```

Autouse Fixtures

```
@pytest.fixture(autouse=True)
def reset_state():
    '''Runs automatically before every test, no need to request it.'''
    GlobalCache.clear()
    yield
    GlobalCache.clear()    # cleanup after too

def test_one():
    # reset_state already ran before this
    assert GlobalCache.is_empty()
```

Multiple Teardown Steps

```
@pytest.fixture
def resource():
    print('Setup A')
    res = acquire_resource()
    yield res
    print('Teardown A')    # always runs, even on test failure
    res.release()

# finalizers via request.addfinalizer (alternative to yield)
@pytest.fixture
def resource_v2(request):
    res = acquire_resource()
```



```
request.addfinalizer(res.release)  
return res
```

6. conftest.py & Fixture Sharing

conftest.py is a special file pytest automatically discovers. Fixtures defined there are shared across all tests in that directory and subdirectories — no imports needed.

```
# Project structure:
# project/
#   conftest.py      <- shared fixtures (root level)
#   tests/
#     conftest.py    <- fixtures for tests/ only
#     test_users.py
#     test_orders.py

# conftest.py
import pytest

@pytest.fixture
def client():
    from myapp import create_app
    app = create_app(testing=True)
    with app.test_client() as c:
        yield c

@pytest.fixture
def db():
    database = setup_test_db()
    yield database
    teardown_test_db(database)

# test_users.py – no import required!
def test_create_user(client, db):
    response = client.post('/users', json={'name': 'Alice'})
    assert response.status_code == 201
```

✓ **Tip:** Use multiple conftest.py files at different directory levels to scope fixtures appropriately.

7. Parametrization

Run the same test logic with multiple sets of input data, avoiding repetitive test functions.

Basic Parametrize

```
import pytest

@pytest.mark.parametrize('a, b, expected', [
    (2, 3, 5),
    (-1, 1, 0),
    (0, 0, 0),
    (100, 200, 300),
])
def test_add(a, b, expected):
    assert add(a, b) == expected

# Generates 4 separate test cases:
# test_add[2-3-5], test_add[-1-1-0], test_add[0-0-0], test_add[100-200-300]
```

With Custom IDs

```
@pytest.mark.parametrize('input_val, expected', [
    pytest.param('hello', 'HELLO', id='lowercase'),
    pytest.param('World', 'WORLD', id='mixed_case'),
    pytest.param('', '', id='empty_string'),
])
def test_upper(input_val, expected):
    assert input_val.upper() == expected
```

Stacking Parametrize (Cartesian Product)

```
@pytest.mark.parametrize('x', [1, 2])
@pytest.mark.parametrize('y', [10, 20])
def test_combo(x, y):
    assert x + y > 0

# Runs 4 times: (1,10) (1,20) (2,10) (2,20)
```

Parametrized Fixtures

```
@pytest.fixture(params=['sqlite', 'postgres', 'mysql'])
def db_backend(request):
    return request.param

def test_connection(db_backend):
    conn = connect(db_backend)
    assert conn.is_active

# Test runs once for each backend automatically
```

■ **Note:** Marking expected failures within parametrize: use `pytest.param(..., marks=pytest.mark.xfail)`.

8. Markers

Markers tag tests with metadata — used for skipping, filtering, expected failures, and custom organisation.

Built-in Markers

```
import pytest

# Skip unconditionally
@pytest.mark.skip(reason='Not implemented yet')
def test_future_feature():
    ...

# Skip conditionally
import sys
@pytest.mark.skipif(sys.version_info < (3, 10),
                    reason='Requires Python 3.10+')
def test_new_syntax():
    ...

# Expected to fail
@pytest.mark.xfail(reason='Known bug, ticket #123')
def test_buggy_feature():
    assert buggy_function() == 'fixed'

# xfail that's allowed to pass unexpectedly
@pytest.mark.xfail(strict=False)
def test_flaky():
    ...

# Timeout (requires pytest-timeout plugin)
@pytest.mark.timeout(5)
def test_must_be_fast():
    ...
```

Custom Markers

```
# pyproject.toml or pytest.ini – register markers
# [tool.pytest.ini_options]
# markers = [
#     'slow: marks tests as slow',
#     'integration: marks integration tests',
# ]

@pytest.mark.slow
def test_large_dataset_processing():
    ...

@pytest.mark.integration
def test_external_api_call():
    ...

# Run only marked tests:
# pytest -m slow
# Exclude marked tests:
# pytest -m 'not slow'
```

```
# Combine:  
# pytest -m 'integration and not slow'
```

9. Testing Exceptions

```
import pytest

def divide(a, b):
    if b == 0:
        raise ValueError('Cannot divide by zero')
    return a / b

# Basic exception test
def test_divide_by_zero():
    with pytest.raises(ValueError):
        divide(10, 0)

# Check exception message
def test_divide_by_zero_message():
    with pytest.raises(ValueError, match='Cannot divide'):
        divide(10, 0)

# Capture exception info for further inspection
def test_exception_details():
    with pytest.raises(ValueError) as exc_info:
        divide(10, 0)
    assert 'zero' in str(exc_info.value)
    assert exc_info.type is ValueError

# Multiple possible exception types
def test_multiple_exceptions():
    with pytest.raises((ValueError, TypeError)):
        divide(10, 'a')

# Testing warnings
def test_deprecation_warning():
    with pytest.warns(DeprecationWarning):
        old_function()
```

10. Mocking & Patching

Mocking replaces real objects/functions with fake ones during tests — useful for isolating code from external dependencies (APIs, databases, file systems).

unittest.mock Basics

```
from unittest.mock import Mock, MagicMock, patch

# Mock object
mock_obj = Mock()
mock_obj.method.return_value = 42
print(mock_obj.method())      # 42
mock_obj.method.assert_called_once()

# Mock with side_effect (raise exception or sequence)
mock_obj.fetch.side_effect = ConnectionError('timeout')
mock_obj.counter.side_effect = [1, 2, 3]  # different value each call
```

patch — Replace Real Objects

```
# my_module.py
import requests

def get_user(user_id):
    response = requests.get(f'https://api.example.com/users/{user_id}')
    return response.json()

# test_my_module.py
from unittest.mock import patch

@patch('my_module.requests.get')
def test_get_user(mock_get):
    mock_get.return_value.json.return_value = {'id': 1, 'name': 'Alice'}

    result = get_user(1)

    assert result == {'id': 1, 'name': 'Alice'}
    mock_get.assert_called_once_with(
        'https://api.example.com/users/1'
    )

# As a context manager
def test_get_user_cm():
    with patch('my_module.requests.get') as mock_get:
        mock_get.return_value.json.return_value = {'id': 2}
        assert get_user(2) == {'id': 2}
```

monkeypatch Fixture (pytest native)

```
# monkeypatch auto-undoes changes after test – safer than manual patch

def test_env_var(monkeypatch):
    monkeypatch.setenv('API_KEY', 'test-key-123')
    import os
    assert os.environ['API_KEY'] == 'test-key-123'
```

```
def test_replace_function(monkeypatch):
    def fake_get_time():
        return '2024-01-01 00:00:00'

    monkeypatch.setattr('my_module.get_current_time', fake_get_time)
    assert my_module.get_current_time() == '2024-01-01 00:00:00'

def test_replace_attribute(monkeypatch):
    monkeypatch.setattr(requests, 'get', lambda url: FakeResponse())

def test_delete_attribute(monkeypatch):
    monkeypatch.delattr('os.getcwd')
```

✓ **Tip:** Prefer monkeypatch over unittest.mock.patch in pytest tests — it integrates with fixtures and auto-cleans up.

11. Testing Class Based Code

```
# shopping_cart.py
class ShoppingCart:
    def __init__(self):
        self.items = []

    def add(self, item, price):
        self.items.append({'item': item, 'price': price})

    def total(self):
        return sum(i['price'] for i in self.items)

# test_shopping_cart.py
import pytest
from shopping_cart import ShoppingCart

class TestShoppingCart:

    @pytest.fixture
    def cart(self):
        '''Fresh cart for every test method.'''
        return ShoppingCart()

    def test_empty_cart_total(self, cart):
        assert cart.total() == 0

    def test_add_item(self, cart):
        cart.add('Book', 15.99)
        assert len(cart.items) == 1
        assert cart.total() == 15.99

    def test_multiple_items(self, cart):
        cart.add('Book', 15.99)
        cart.add('Pen', 2.50)
        assert cart.total() == pytest.approx(18.49)
```

12 Temporary Files & Directories

```
def test_write_to_tmp_file(tmp_path):
    '''tmp_path is a unique pathlib.Path per test, auto cleaned up.'''
    file = tmp_path / 'data.txt'
    file.write_text('Hello, World!')

    assert file.read_text() == 'Hello, World!'
    assert file.exists()

def test_create_directory_structure(tmp_path):
    sub_dir = tmp_path / 'config'
    sub_dir.mkdir()
    config_file = sub_dir / 'settings.json'
    config_file.write_text('{"debug": true}')

    assert config_file.exists()
    assert sub_dir.is_dir()

# tmp_path_factory – for module/session-scoped temp dirs
@pytest.fixture(scope='module')
def shared_tmp_dir(tmp_path_factory):
    return tmp_path_factory.mktemp('shared_data')

def test_using_shared_dir(shared_tmp_dir):
    f = shared_tmp_dir / 'output.csv'
    f.write_text('a,b,c')
    assert f.exists()
```

13. Capturing Output & Logs

capsys - Capture stdout/stderr

```
def greet(name):
    print(f'Hello, {name}!')

def test_greet(capsys):
    greet('Alice')
    captured = capsys.readouterr()
    assert captured.out == 'Hello, Alice!\n'
    assert captured.err == ''
```

caplog - Capture Logging

```
import logging

def process_order(order_id):
    logging.info(f'Processing order {order_id}')
    if order_id < 0:
        logging.error('Invalid order ID')
        raise ValueError('Invalid order')

def test_process_order_logs(caplog):
    with caplog.at_level(logging.INFO):
        process_order(123)
    assert 'Processing order 123' in caplog.text

def test_process_order_error_log(caplog):
    with pytest.raises(ValueError):
        process_order(-1)
    assert 'Invalid order ID' in caplog.text
    assert caplog.records[0].levelname == 'ERROR'
```

14. Plugins

Pytest's plugin ecosystem extends it with hundreds of useful features.

Plugin	Purpose
pytest-cov	Code coverage reports
pytest-xdist	Parallel test execution
pytest-mock	Cleaner mocker fixture (wraps unittest.mock)
pytest-asyncio	Test async/await code
pytest-django	Django-specific test helpers
pytest-html	Generate HTML test reports
pytest-timeout	Fail tests that run too long
pytest-randomly	Randomize test order (catches hidden deps)
pytest-rerunfailures	Auto-retry flaky tests
pytest-playwright	Browser automation testing

```
pip install pytest-mock
```

```
# Using pytest-mock's 'mock' fixture
def test_with_mock(mock):
    mock_get = mock.patch('my_module.requests.get')
    mock_get.return_value.json.return_value = {'status': 'ok'}
    assert my_module.check_status() == 'ok'
# No manual cleanup needed – mock handles it
```

15. Test Organization

```
# Recommended project layout
myproject/
  src/
    myproject/
      __init__.py
      calculator.py
  tests/
    __init__.py
    conftest.py          # shared fixtures
    unit/
      test_calculator.py
    integration/
      conftest.py        # integration-specific fixtures
      test_api.py
  pyproject.toml
  pytest.ini
```

Grouping with Classes vs Modules

```
# Group related tests in a class
class TestUserValidation:
    def test_valid_email(self): ...
    def test_invalid_email(self): ...
    def test_empty_name(self): ...

class TestUserPermissions:
    def test_admin_access(self): ...
    def test_guest_denied(self): ...
```

✓ **Tip:** Keep unit tests (fast, isolated) separate from integration tests (slower, real dependencies) using directories and markers.

16 Coverage

```
pip install pytest-cov

# Run tests with coverage report
pytest --cov=myproject

# Show missing lines
pytest --cov=myproject --cov-report=term-missing

# Generate HTML report
pytest --cov=myproject --cov-report=html
# Opens htmlcov/index.html

# Fail if coverage below threshold
pytest --cov=myproject --cov-fail-under=80

# Exclude files from coverage (.coveragerc or pyproject.toml)
# [tool.coverage.run]
# omit = ['*/tests/*', '*/migrations/*']
```

17 Async Testing

```
pip install pytest-asyncio

import pytest
import asyncio

async def fetch_data():
    await asyncio.sleep(0.1)
    return {'status': 'success'}

@pytest.mark.asyncio
async def test_fetch_data():
    result = await fetch_data()
    assert result['status'] == 'success'

# Async fixtures
@pytest.fixture
async def async_client():
    client = await create_async_client()
    yield client
    await client.close()

@pytest.mark.asyncio
async def test_with_async_fixture(async_client):
    response = await async_client.get('/health')
    assert response.status == 200

# pyproject.toml – set default mode so you don't need the marker
# [tool.pytest.ini_options]
# asyncio_mode = 'auto'
```

18. Configuration Files

`pyproject.toml` (recommended)

```
[tool.pytest.ini_options]
minversion = '7.0'
addopts = '-ra -q --strict-markers --cov=src'
testpaths = ['tests']
python_files = ['test_*.py']
python_classes = ['Test*']
python_functions = ['test_*']
markers = [
    'slow: marks slow-running tests',
    'integration: marks integration tests',
]
filterwarnings = [
    'error',
    'ignore::DeprecationWarning',
]
```

`pytest.ini` (alternative)

```
[pytest]
minversion = 7.0
addopts = -ra -q
testpaths = tests
markers =
    slow: marks slow-running tests
```

`setup.cfg` (legacy)

```
[tool:pytest]
testpaths = tests
addopts = --strict-markers
```


19. Hooks

Hooks let you customise pytest's behaviour at various points in the test lifecycle. Define them in `conftest.py`.

```
# conftest.py

def pytest_configure(config):
    '''Runs once before test collection begins.'''
    print('Starting test session')

def pytest_collection_modifyitems(config, items):
    '''Modify/reorder collected tests.'''
    for item in items:
        if 'slow' in item.keywords:
            item.add_marker(pytest.mark.skip(reason='slow tests disabled'))

def pytest_runtest_setup(item):
    '''Runs before each test.'''
    print(f'Setting up: {item.name}')

def pytest_runtest_teardown(item):
    '''Runs after each test.'''
    print(f'Tearing down: {item.name}')

def pytest_sessionfinish(session, exitstatus):
    '''Runs once after the entire test session.'''
    print(f'Session finished with status: {exitstatus}')

# Custom command-line option
def pytest_addoption(parser):
    parser.addoption('--env', default='staging', help='Target environment')

@pytest.fixture
def env(request):
    return request.config.getoption('--env')

# Usage: pytest --env=production
```

20. Best Practices

Do's

- ✓ Keep tests small, focused, and independent of each other
- ✓ Use fixtures for setup/teardown instead of duplicating code
- ✓ Use descriptive test names: `test_login_fails_with_wrong_password`
- ✓ Prefer plain assert statements — let pytest's introspection help
- ✓ Use `parametrize` instead of copy-pasting near-identical tests
- ✓ Use `monkeypatch`/mockers fixtures rather than manual `try/finally` patches
- ✓ Run tests in random order occasionally to catch hidden dependencies
- ✓ Separate fast unit tests from slow integration tests with markers

Don'ts

- ✗ Don't let tests depend on execution order or shared mutable state
- ✗ Don't use `time.sleep()` to wait for async results — use proper awaits
- ✗ Don't test implementation details — test behaviour/contracts
- ✗ Don't ignore flaky tests — fix or quarantine them with markers
- ✗ Don't put real network/database calls in unit tests
- ✗ Don't overuse mocks — mocking everything reduces real test value

Quick Reference

Concept	Syntax	Purpose
Fixture	<code>@pytest.fixture</code>	Reusable setup/teardown
Parametrize	<code>@pytest.mark.parametrize</code>	Run test with multiple inputs
Skip	<code>@pytest.mark.skip</code>	Skip a test
Expected fail	<code>@pytest.mark.xfail</code>	Mark test as expected to fail
Raises	<code>pytest.raises(Err)</code>	Assert exception is raised
Approx	<code>pytest.approx(val)</code>	Compare floats with tolerance
Mock	<code>monkeypatch</code> / <code>mock</code>	Replace real objects in tests
Temp dir	<code>tmp_path</code>	Unique per-test temp directory
Capture stdout	<code>capsys</code>	Assert on printed output
Capture logs	<code>caplog</code>	Assert on logging output